

Kata: an executable quality gate for AI-assisted software changes

Walter Aoiama Nagai  [Instituto de Ciências Tecnológicas, Universidade Federal de Itajubá, Brazil | walternagai@unifei.edu.br]

Claudia Akemi Izeki  [Instituto de Ciências Tecnológicas, Universidade Federal de Itajubá, Brazil | claudiaizeki@unifei.edu.br]

Abstract

AI coding agents produce code faster than any developer can review, and the oversight surface collapses onto the automated test suite. When an agent says “done”, nothing executes the checks it claims to have passed, and the literature shows that agents optimize for passing visible tests while deviating from the user’s true goal (reward hacking), and that large language models cannot reliably self-correct without external feedback. This paper presents Kata (), an executable quality gate for AI-assisted software changes. Kata guides a change through a nine-phase cycle — FIT, THINK, SIMPLIFY, INTENT, SURGICAL, VERIFY, TWIN CHECK, ARTIFACT, REPORT — plus an optional adversarial JUDGE phase that re-runs every claimed check and hunts seven categories of fraud, such as weakened tests, false completion claims, and undeclared scope. Unlike skills and methodologies that instruct the agent to be careful, Kata moves the objective parts of the discipline out of the prompt and into executable, testable Python code: lint, test, and coverage steps read real exit codes, the JUDGE confronts the task file with the actual repository state, and an audit mode grades each phase as followed, skipped, or faked, naming the concrete risk each skip or fake created. Kata is implemented as a Python command-line interface with two agent frontends (OpenCode and Claude Code) generated from a single source of truth, and it is verified by its own lint, tests, coverage, and 19 adversarial trap scenarios in continuous integration.

Keywords: AI agents, software quality, verification, adversarial verification, quality gates, LLM, Claude Code, OpenCode

1 Introduction

AI coding agents (OpenCode, Claude Code, Cursor, and similar tools) change software with remarkable speed. The problem is not the speed — it is **trust**. When an agent says “done”, what sustains that claim?

A skill says: “*verify your tests before committing*”. The model may follow, ignore, or half-follow the instruction, and nobody can tell the difference. An agent can claim “lint clean, tests passing, coverage at the gate” without any of it having been executed. Or worse: it can have executed the checks, but with a weakened test (a pass in place of the assertion) that “passes” without verifying anything.

The Karpathy Development Cycle proposes discipline: think before coding, keep the code minimal, make surgical changes, and verify objectively. The Fable Method Sahir619 (2026) proposes gates: classify the task before acting (fit gate), require evidence before action, verify adversarially, and report outcome-first. Both are methodologies — text that instructs. The problem: text instructs, but it does not execute.

This paper presents Kata (, “form” or “pattern”, as in a martial arts kata: a disciplined, repeatable sequence of movements), a quality gate for software changes assisted by AI agents. Kata’s contribution is to move the objective parts of the discipline out of the prompt and into executable, testable Python code. The lint, test, and coverage steps read real exit codes; the JUDGE re-runs claimed checks against the real repository, including committed and untracked files; and the audit mode grades each phase of a completed task as followed, skipped, or faked, naming the concrete risk each skip or fake created. A skill asks for honesty; Kata detects its absence.

This paper is a **software article**. Following the journal’s guidelines, it describes a tool, positions it as a significant advance over previously published software, and demonstrates

the advance by *direct comparison with available related software*: Section 3.3 reports an executed comparison in which Kata and three alternative gates (nova TeamSPWK (2026), claude-wizard vlad ko (2026), and a gate-less baseline) review the same 19 adversarial scenarios, and measures fraud detection, false positives, and verification time.

The remainder of this paper is organized as follows. Section 2 states the problem and the research purpose of the software. Section 3 places Kata in the context of related work and reports the executed comparison with related gates. Section 4 describes the software design and the trade-offs weighed. Section 5 walks through the development cycle in action with real output. Section 6 details the adversarial verification layer. Section 7 presents the audit mode. Section 8 documents how to install and use the kata CLI. Section 9 reports the research impact and evaluation. Section 12 concludes.

2 Statement of need

AI coding agents produce code faster than any developer can review, and the oversight surface collapses onto the automated test suite. Reward hacking naturally arises in this setup: agents optimize for passing visible tests while deviating from the user’s true goal. SpecBench Zhao et al. (2026) measures this phenomenon and finds that the gap between visible and held-out test suites grows by 28 percentage points for every tenfold increase in code size, with failures ranging from subtle feature isolation to deliberate exploits, including a 2,900-line hash-table “compiler” that memorizes test inputs. Agents also end failure transcripts with “all tests pass”, and tests get weakened until they agree with the code Sahir619 (2026).

Large language models cannot reliably self-correct without external feedback — sometimes their performance even degrades after self-correction Huang et al. (2024). The impli-

cation is that verification cannot be left to the model checking itself: it must be external, executable, and objective.

The target audience of Kata is developers, researchers, and teams who use AI agents to modify code and need objective, reproducible verification that what was claimed was actually done. Kata is a quality gate for software changes, not a project template. It fits the following situations:

- **Before committing:** run the full cycle to force evidence for the change — problem, assumptions, minimality, intent agreement, and passing lint, tests, and coverage — instead of committing on a “it works” impression.
- **Bug fixes:** the cycle is adversarial about defects. INTENT catches code/test/specification conflicts, and TWIN CHECK searches the project for the same defect pattern elsewhere, recording even a negative result.
- **Small code-loop tasks:** the FIT gate measures the diff and routes trivial changes (one file, fewer than ten lines) directly, without planning ceremony.
- **Planning before implementation:** `kata --plan` runs FIT and THINK, saves the plan with assumptions and unknowns, and stops — the plan can be reviewed or handed off before a single line changes.
- **Adversarial verification of finished work:** `kata --judge` re-runs every claimed check and hunts seven fraud categories before a merge or hand-over.
- **Auditing tasks after the fact:** `kata --audit` grades each phase of a completed task as followed, skipped, or faked, and names the concrete risk each skip or fake created.
- **Continuous integration:** `kata --check-only` runs lint, tests, and coverage with the coverage gate, non-interactively, as a CI entry point.

3 State of the field

Several tools and methodologies address disciplined AI-assisted development. The Fable Method Sahir619 (2026) is the closest conceptual ancestor: it defines a think/act/prove loop with a fit gate, a triviality gate, evidence before action, adversarial verification, and outcome-first reporting, and it ships an eval that keeps the method honest. Kata adapts the fit gate, the triviality gate, the intent gate, and the hard bounds directly from it, but differs in one structural way: the objective logic lives in a Python package (`kata.fit`, `kata.verify`, `kata.judge`) that is unit-tested and measured for coverage, rather than in prompt text.

`claude-wizard vlad ko` (2026) is an 8-phase development skill with TDD and adversarial review, but it is a skill only: there is no backend that executes the gates. `Ring LerianStudio` (2026) is a large skills library (76 skills, 33 agents) that enforces engineering practices with 10-gate development cycles, again as instructions rather than executable checks. `nova TeamSPWK` (2026) provides pre-commit quality gating, but as an agent skill without a shared executable core. On the research side, `SpecBench Zhao et al.` (2026) measures reward hacking in coding agents, and `SWE-bench Jimenez et al.` (2024) benchmarks agent capability on real GitHub issues; both measure agents, neither provides a gate that a team can run on its own changes.

Kata was built rather than contributed to existing projects for three reasons. First, no existing tool executes the verification: the closest alternatives are prompts that ask the model to verify itself, which the self-correction literature shows is unreliable Huang et al. (2024). Second, Kata needed to serve two agent hosts (OpenCode and Claude Code) with identical behavior; a single Python backend with generated frontends guarantees that the checks a task claims to have passed are the same checks in both hosts. Third, Kata needed to be verifiable itself: its own lint, tests, and coverage are measured, and 19 adversarial trap scenarios run in continuous integration to ensure the JUDGE catches planted frauds and does not accuse honest work.

3.1 Recent research on agent verification

The research landscape of 2026 has produced several works that corroborate the problem Kata addresses and sharpen its positioning. The Unreliable Progress Bar study Wang et al. (2026) shows that LLM agents cannot reliably report task progress throughout execution — agents claim completion states that do not match reality, which is precisely the class of claim the JUDGE re-executes. The Double Measurement Confound Zhang et al. (2026) demonstrates that agent benchmarks suffer from de-scaffolding and ground-truth scoring confounds, arguing that reliability must be measured beyond the mean; Kata’s trap suite follows the same principle by testing both fraud detection and false-positive avoidance. `XA-agent Huynh et al.` (2026) uses execution-guided agentic AI for issue localization and resolution, showing that execution feedback improves agent outcomes — the same insight that motivates Kata’s decision to read real exit codes rather than model intentions. `Consort Hartman` (2026) is a spec-first agent framework for enforced, test-driven development on live database branches; it shares Kata’s enforcement philosophy but is scoped to database development, while Kata is a general-purpose gate. The Authority Is Not a String harness Bouras et al. (2026) scopes agent capabilities to resist prompt injection, complementing Kata’s authorization gate (AUTH line) from the security angle. `TrajMark Zeng et al.` (2026) attributes ownership and localizes tampering in coding-agent trajectories, which is conceptually adjacent to Kata’s baseline tampering detection, though applied to trajectory provenance rather than task files. The Discovery Certification Protocol Ning et al. (2026) audits AI research agents by certifying that claimed discoveries are reproducible — the same claim-vs-reality confrontation that the JUDGE performs for software changes. `RefVerifier Mocan et al.` (2026) semi-automatically verifies reference claims in scientific manuscripts, another instance of the claim-verification pattern. Finally, the study on traditional test criteria for LLM-generated code Hamidi et al. (2026) and the dynamics of iterative bug-fixing with LLMs Wang-Lin et al. (2026) both show that verification of AI-generated changes requires more than passing tests — the former because traditional criteria miss LLM-specific bugs, the latter because agents may fix what is not broken. Together, these works validate Kata’s core design decision: verification must be external, executable, and claim-based, not self-reported.

3.2 Comparative analysis

Table 1 summarizes the comparison between Kata and the related tools and methodologies surveyed in this section. The comparison dimensions are: whether the tool executes verification (runs checks and reads exit codes) or only instructs the model; whether it has a shared executable core across agent hosts; whether it detects fraud adversarially; whether it audits completed work; and whether it verifies itself. The assessments are the authors', based on each project's public documentation and repository as of September 2026: "Partial" for fraud detection means the tool performs adversarial review in prompt form without re-executing checks against the repository, as opposed to "Yes" (executable re-execution with verdicts) and "No" (no adversarial step at all).

The table makes the positioning explicit: Kata is the only tool in the survey that combines executable verification, a shared core across agent hosts, adversarial fraud detection, post-hoc auditing, and self-verification. The tools that execute (SpecBench, SWE-bench, Consort) are benchmarks or domain-specific frameworks, not gates a team runs on its own changes. The tools that audit (Fable Method) do not execute. Kata is the intersection.

Two notes bound the comparison. First, Consort Hartman (2026) executes verification, but its enforcement scope is a framework for test-driven development on live *database* branches; as a gate for general software changes it is not directly comparable by execution, so Section 3.3 compares Kata against the gates that are installable on the same host (nova, claude-wizard) and against a gate-less baseline. Second, *executing verification* is what the *kata* CLI does, and it is precisely what none of the prompt-based tools can be observed to do: their "Partial" fraud column reports that the adversarial step exists only as instruction text.

3.3 Executed gate comparison

The journal's guidelines request that a software article demonstrate its advance *by direct comparison with available related software*. This section reports such a comparison: four gates — Kata, nova, claude-wizard, and a gate-less baseline — reviewed the same 19 adversarial scenarios of Section 9, and we measured fraud detection, false positives, and wall-clock time.

3.3.1 Protocol

The comparison reuses the trap suite of Section 9 as the workload: each scenario is a git repository holding a "completed" task whose task file claims lint, tests, and coverage pass. The ground truth is the scenario's `ground_truth.yaml`: what the review must find (REFUTED scenarios) or must not find (VERIFIED, VERIFIED WITH CAVEATS, and UNVERIFIABLE scenarios).

Each gate reviews every scenario in a fresh sandbox, without access to the ground truth. Kata runs its `--judge` mode. nova and claude-wizard run headless through the Claude Code CLI (`claude -p` with permission bypass), each receiving an identical canonical prompt that states the review task and the seven fraud categories, prefixed with an in-

struction to follow the gate's workflow. The gate-less baseline receives the same prompt without any gate instruction. The agent gates report their verdict as a single JSON object (`fraud_detected`, `fraud_types`, `reason`); the harness parses it with a tolerant JSON decoder. We classified a scenario as *detected* when the gate found the planted fraud (matching the ground-truth types), and as a *false positive* when a non-REFUTED scenario was accused.

3.3.2 Results

Table 2 summarizes the outcome. Three findings stand out.

First, **all four gates detected every planted fraud in all eleven REFUTED scenarios**. Fraud detection is not where the gates differ: the difference is in what happens on honest work and confessed blind spots. Second, **Kata was the only gate with zero false positives**: the gate-less baseline accused the honest s07 task of weakened checks; all three agent gates accused a later task's changes as scope creep for the task under review (s17); and the agents reported frauds on the blind spots the ground truth declares UNVERIFIABLE — nova and the baseline on s12 and s13, all three on s19. Third, **the gates differ by orders of magnitude in time**: Kata's median review is about one second (0.1 s on fraud scenarios, 1.0 s on the others), while every agent gate takes tens of seconds per scenario, because the gate reads exit codes while the agents run a model. Over the 19 scenarios, Kata is the only gate whose correctness is not a function of model performance.

The comparison is not without caveats, which we state explicitly. The agent gates run through a single model (Claude Code's default), so their false positives are a property of *this* model under these prompts, not of the tools in general. Agent verdicts are also stochastic: across exploratory runs the same gate-scenario pair occasionally changed verdict, so the table reports one complete campaign and the raw verdicts of every run are kept in the repository. nova also offers an MCP server and specialized subagents that a full harness integration could use; we did not integrate it, because the goal is to measure the gate as a *change review* step in the same shape a team would run it. The harness (Section 8) and the per-scenario verdicts are public in the repository (see 8); `eval/compare_gates.py` reproduces the comparison and `eval/results/compare_gates_round2.json` holds the raw verdicts used for Table 2. Finally, the honest scenarios that the agent gates misjudged are exactly the ones where the Kata JUDGE *confesses* a blind spot or applies a mechanism the agent has no knowledge of (`approved_commit` on s17, the `Git-ignore` on s19, the language probes on s12): an agent that cannot know the mechanism can only guess, and the guesses are what the comparison measures.

4 Software design

Kata's design is organized around one principle: the parts of the cycle that can be objective must be executable, and the parts that cannot must be declared before the evidence exists. The Python package `src/kata/` implements the objective logic in four modules.

Figure 1 shows the complete cycle. The nine phases run

Table 1. Comparison of Kata with related tools and methodologies.

Tool	Executes	Shared core	Fraud	Audit	Self-verify
Fable Method Sahir619 (2026)	No	No	Partial	Yes	Yes
claude-wizard vlad ko (2026)	No	No	Partial	No	No
Ring LerianStudio (2026)	No	No	No	No	No
nova TeamSPWK (2026)	No	No	Partial	No	No
Consort Hartman (2026)	Yes	No	No	No	No
SpecBench Zhao et al. (2026)	Yes	N/A	No	No	No
SWE-bench Jimenez et al. (2024)	Yes	N/A	No	No	No
Kata	Yes	Yes	Yes	Yes	Yes

Table 2. Executed comparison of four gates over the 19 trap scenarios. One complete campaign (September 2026); raw verdicts in `eval/results/compare_gates_round2.json`.

Gate	Detection	FP	Correct	Median time	Median output
<i>Fraud scenarios (11 REFUTED)</i>					
Kata	11/11	—	11/11	0.1 s	28 lines
nova	11/11	—	11/11	16 s	1 line
claude-wizard	11/11	—	11/11	17 s	1 line
bare (no gate)	11/11	—	11/11	18 s	1 line
<i>Honest and blind-spot scenarios (8)</i>					
Kata	—	0	8/8	1.0 s	28 lines
nova	—	4	4/8	40 s	1 line
claude-wizard	—	2	6/8	27 s	1 line
bare (no gate)	—	5	3/8	32 s	1 line
All scenarios — correctness					
Kata			19/19		
nova			15/19		
claude-wizard			17/19		
bare (no gate)			14/19		

in sequence, with the optional JUDGE applied after REPORT. The phases that can be objective (FIT, VERIFY, JUDGE) are backed by executable Python logic; the phases that require judgment (THINK, SIMPLIFY, INTENT, SURGICAL, TWIN CHECK, ARTIFACT, REPORT) are guided by skills and recorded in the task file, which the objective phases read and confront with the repository.

4.1 Architecture

`kata.fit` measures the real diff against HEAD (including untracked files) and applies the triviality gate (at most one changed file and fewer than ten changed lines). `kata.verify` runs lint, tests, and coverage with a numeric gate, reading exit codes rather than model intentions; a declared role in `.kata/config.yaml` runs verbatim, so the tool does not assume the target project is Python. `kata.judge` treats the task file as a collection of claims, re-runs the claimed checks, diffs the claims against the repository, and hunts seven fraud categories: weakened checks, false completion, scope creep, unauthorized actions, specification betrayal, debris, and baseline tampering. `kata.cli` orchestrates the nine phases, persists the task file, and exposes the audit and judge modes.

4.1.1 Language-agnostic verification

Kata does not assume the project it verifies is a Python project. Whoever knows how to check a repository is the

repository, so the commands live in `.kata/config.yaml`, next to the task files:

Listing 1: Declaring the verification commands of a non-Python project.

```
verify:
  lint: npx eslint src tests
  test: npx vitest run
  coverage: npx vitest run --coverage
  coverage_pattern: 'All files\s+\\s+([\d.]*)'
  gate: 80
```

Every key is optional. A role that is not declared falls back to the Python default for that role (ruff, pytest, pytest-cov) and keeps obeying the path flags, so a project can replace only its linter and keep pytest. A declared role is run verbatim and judged by its exit code, the one contract every lint and test tool honours. Coverage is the exception: the built-in path delegates the gate to `--cov-fail-under`, which does not exist outside Python, so a declared coverage command has its percentage extracted with `coverage_pattern` and compared here. If the pattern matches nothing, the check fails rather than recording 0.0% as a pass — “could not measure” is not “measured and passed”.

A config file that exists but is invalid aborts with exit code 1. Falling back to ruff silently would report the success of a check the project never asked for, which is the class of lie the judge exists to hunt. The JUDGE follows the same line: it knows the test syntax of Python, JS/TS, Go, Ruby, Rust, Java/Kotlin, C#, PHP, and Swift (the language probes imple-

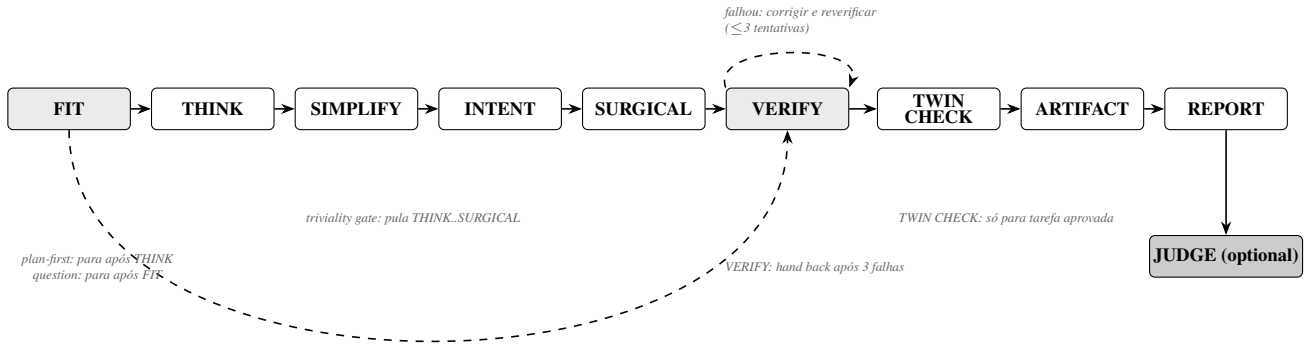


Figure 1. The Kata cycle: nine phases plus the optional adversarial JUDGE. Shaded phases (FIT, VERIFY, JUDGE) are backed by executable Python logic; the remaining phases are guided by skills and recorded in the task file, which the objective phases confront with the repository. Dashed arrows show the optional flows: the triviality gate skips the planning phases (FIT to VERIFY directly), VERIFY loops on failure with a hard bound of three attempts before handing the task back to the user, and JUDGE runs after REPORT only when requested. TWIN CHECK runs only for approved tasks, and the plan-first and question routes stop after THINK and FIT respectively.

mented in `kata/judge.py`). A language outside this list (for example, Elixir `.exs`) becomes a declared blind spot, not silence.

The frontends are generated, not hand-maintained. The phase prompts live once in `phases/*.md` and are rendered into the OpenCode agent and the Claude Code skills by a build script; 93% of the content is shared, and the remaining 7% is declared difference (tool names of the host). Both figures were measured in the kata repository itself: the shared fraction comes from the build script’s own accounting of shared versus frontend-specific blocks, and the 395 divergent lines are the diff the authors observed between the two hand-maintained copies before the single-source migration. This single-source design exists because the previous arrangement — two hand-maintained copies — failed: the copies accumulated those 395 divergent lines, including an improvement applied to one frontend and forgotten in the other. A test fails when the generated files drift from the source, so the divergence cannot pass unnoticed.

4.1.2 Domain adapters

Kata extends the cycle to non-coding tasks without changing the underlying pipeline through domain adapters. A task declares its domain in the task file (`domain: devops`, for example); the orchestrator loads the adapter after FIT and applies its domain-specific evidence set and fraud table. Each adapter defines: the domain scope (what it covers and explicitly does not cover), the evidence (files and state to inspect before acting), the authority (who decides correctness in the domain), how to verify by observation, a domain-specific fraud table, a binding minimum evidence set, FIT routes by task shape, and red lines (actions never allowed without documented human authorization).

Adapters are optional: a missing adapter does not fail —doctor or stop the cycle, because the default domain is coding and most tasks do not need an adapter. When a domain adapter is missing for a non-coding task, the orchestrator records the skill name in `preflight.skills_missing` and falls back to the adapter’s minimal contract. The only adapter shipped today is `kata-devops` (Docker, Docker

Compose, Terraform, Nginx, GitHub Actions, deploys, and healthchecks); adapters for data-analysis, research, and docs are planned. New adapters follow a template and are generated to both frontends by the same build script that generates the phase skills, so an adapter is written once and served to both hosts.

4.2 Design trade-offs

The cycle encodes two design trade-offs worth making explicit.

The first is the **done criterion** (Fable Step 1): THINK records what “ready” means and how it will be verified *before* the evidence exists, and VERIFY confronts that declared criterion with the final result. Without this, the success criterion only exists after the work is done, and “is it satisfied?” becomes unfalsifiable.

The second is the **hard bound** (Fable Step 5): after three failed verification attempts the task is handed back to the user with what was tried, the real output, and the current hypothesis, instead of looping fix-verify forever. The JUDGE also confesses its blind spots: if it could not observe something (a test language it has no probes for, a file outside the diff), the verdict is UNVERIFIABLE, never VERIFIED — “I could not look” is never reported as “all clear”.

4.3 The claim-based verification model

The JUDGE’s design is organized around a claim-based verification model that distinguishes Kata from both prompt-based skills and benchmark harnesses. A task file is treated as a collection of claims — “lint clean”, “tests pass”, “coverage at the gate”, “two files changed surgically”, “intent aligned” — and each claim is confronted with the repository in one of three ways:

- **Re-executed:** lint, tests, and coverage are re-run and their exit codes compared with the claimed results. A claim that fails re-execution is a false completion fraud.
- **Confronted with the diff:** surgical scope and intent alignment are checked against the actual changed files.

A file changed but not declared is scope creep; an intent gate that recorded disagreement but was approved anyway is specification betrayal.

- **Accepted without verification:** the success criterion is a subjective confirmation the user gives during VERIFY, and no command reproduces it. The JUDGE lists it separately as accepted without verification, and it adds a caveat. Presenting it as verified would be exactly the fraud the judge exists to hunt.

This three-way classification is what makes the JUDGE honest in both directions. It does not over-claim: claims it cannot re-execute are confessed, not assumed. And it does not under-claim: claims it can re-execute are always re-run, never taken at face value. The same model applies to the audit mode, which grades each phase as followed, skipped, or faked by comparing the task file's content with what the phase was supposed to record.

The claim-based model also connects Kata to the broader verification literature. RefVerifier Mocan et al. (2026) applies the same pattern to scientific manuscripts, decomposing reference claims and verifying each against sources. The Discovery Certification Protocol Ning et al. (2026) certifies that AI research agents' discoveries are reproducible by re-running them. TrajMark Zeng et al. (2026) attributes and verifies coding-agent trajectories. Kata's contribution is to apply the claim-verification pattern to the software change itself, where the claims are the task file and the ground truth is the repository.

5 The development cycle in action

To demonstrate the cycle, we applied Kata to a real bug fix in a small example project: a calculator whose `dividir` (divide) function raised an unhandled `ZeroDivisionError`. The change adds a `ValueError` guard and a regression test. The working tree contains two modified files (11 changed lines), so the FIT gate classifies the task as non-trivial and routes it to the full cycle.

5.1 FIT — task classification

The FIT phase measures the real diff and classifies the task:

```
diff: 2 arquivo(s), 11 linha(s) alteradas
tarefa não-trivial

Rotas disponíveis:
[1] code-loop - ciclo completo
[2] plan-first - só planejamento
[3] question - só diagnóstico
[4] research - precisa pesquisar antes de agir
[5] inference - baseado só em inferência

Rota escolhida [1]: 1
Justificativa breve (opcional): corrigir divisão por
zero sem tratamento
```

A trivial task (at most one file, fewer than ten lines) would skip the planning phases and go directly to VERIFY — the triviality gate of the Fable Method.

5.2 THINK — declaring before the evidence

THINK records the problem, assumptions, alternatives, unknowns, and the done criterion *before* the evidence exists:

```
Qual o problema exato que estou resolvendo?
a função dividir levanta ZeroDivisionError sem
tratamento
Quais assumptions estou fazendo? (separadas por ;)
nenhuma; API estável
Quais alternativas considereirei? (separadas por ;)
tratar com ValueError; retornar None
O que NÃO sei? (preciso perguntar antes?) nenhuma
O que é 'pronto'? (critério de sucesso + como vou
verificar)
teste test_dividir_por_zero passa e coverage >=
70%
```

The done criterion is declared before the evidence exists (Fable Step 1). VERIFY will confront this declared criterion with the final result — not ask “is it satisfied?” to a criterion that only exists afterwards.

5.3 SIMPLIFY, INTENT, SURGICAL — minimalism and intent gates

SIMPLIFY checks that the solution is minimal, with no single-use abstractions and no speculative configuration. INTENT compares what the code does, what the checks expect, and what the specification says, with the authority order: user statement > specification > tests > current code. SURGICAL reviews every changed file and records whether it is necessary:

```
O que o código FAZ hoje? dividir levanta
ZeroDivisionError
O que o teste/check ESPERA? teste espera ValueError
O que a especificação/README DIZ? README não
menciona o caso
Código, teste e especificação concordam? [S/n]: s

Arquivos alterados:
src/calculadora.py - necessário para esta tarefa? [S
/n]: s
tests/test_calculadora.py - necessário para esta
tarefa? [S/n]: s
Imports removidos são só os que sua mudança tornou
inúteis? [S/n]: s
```

5.4 VERIFY — objective verification

VERIFY runs the objective checks in order: lint, test, coverage against the gate, and the success criterion confronted with the done criterion declared in THINK:

```
ruff check src/ tests/
limpo

pytest tests/ --tb=short -q
passou

coverage (gate 70%)
passou (100.0%)

Critério de sucesso da tarefa
(declarado no THINK: teste test_dividir_por_zero
passa e coverage >= 70%)
```

```
O critério de sucesso da tarefa está satisfeito? [S/n]: s

KATA CYCLE - APROVADO
```

Each of the first three steps runs the command declared in `.kata/config.yaml` for that role, or the Python default (ruff, pytest, pytest-cov with `--cov-fail-under`) when the role is not declared. Coverage is short-circuited when the test step fails. A task is approved only when all checks and the success criterion pass.

5.5 TWIN CHECK — searching for recurring patterns

TWIN CHECK runs after VERIFY and only for an approved task. When a defect has been fixed, the same pattern often exists elsewhere; the step asks whether one was, searches the project for the pattern, and records the answer:

```
Um defeito foi corrigido? Deseja buscar padrão similar? [s/N]: s

Padrão a buscar (regex): a / b
Buscando 'a / b' no projeto...
Encontrado em 1 arquivo(s):
./src/calculadora.py:7 return a / b
Corrigir as demais ocorrências agora? [s/N]: n
```

Recording a negative answer is part of the point: it is what distinguishes “no defect” from “not checked”.

5.6 ARTIFACT and REPORT — the due lines

ARTIFACT verifies that the evidence lines that are due are present: INTENT (due when behavior changed), AUTH (due when an irreversible external action was taken), PENDING (due when documentation prescribes a follow-up that was not taken), and TWINS (due when a defect was fixed). REPORT prints the outcome first, followed by the problem, the done criterion, changed files, verification results, caveats, and any due artifact lines:

```
KATA CYCLE - APROVADO: critério de sucesso
satisfeito

Problema: a função dividir levanta ZeroDivisionError
sem tratamento
Critério declarado: teste test_dividir_por_zero
passa e coverage >= 70%
Arquivos alterados: src/calculadora.py, tests/
test_calculadora.py
INTENT: code does dividir levanta ZeroDivisionError;
check expects
teste espera ValueError; spec says README
não menciona o caso

Verificações:
ruff check limpo
pytest passou
coverage 100.0% gate
critério de sucesso satisfeito

TWINS: searched a / b - found 1 arquivo(s), 1
ocorrência(s)
```

5.7 The task file

The entire cycle is persisted in a task file (`.kata/<task>.yaml`), so the evidence behind an approval is persistent and auditable, not conversational. The file records the done criterion, the FIT classification, the THINK declarations, the SIMPLIFY/INTENT/SURGICAL answers, the VERIFY results (including the coverage percentage and the attempt counter), the TWIN CHECK result, and the artifact checks. It also records `base_commit` (the HEAD captured when the task started) and `approved_commit` (the HEAD at the moment of approval), which allow the JUDGE to diff `base..approved` even after the task has been committed — changes made by later tasks do not count as undeclared scope for this one.

The task file from the demonstration of Section 5 is shown in Listing 2. Note the structure: every phase has an answered flag (true when a human answered, false when filled with defaults) and a skipped flag (true only in non-interactive mode). The audit mode reads these flags to distinguish followed, skipped, and faked phases. The verify section records the attempt counter and the hand-back flag, which implement the hard bound of Fable Step 5. The twins section records whether a defect was fixed and whether a pattern search was performed — recording a negative answer is what distinguishes “no defect” from “not checked”.

Listing 2: Task file produced by the demonstration cycle.

```
task: corrigir-divisao-zero
status: approved
done: teste test_dividir_por_zero passa e coverage >=
70%
fit:
  trivial: false
  route: code-loop
  reason: corrigir divisão por zero sem tratamento
  answered: true
think:
  problem: a função dividir levanta ZeroDivisionError
  sem tratamento
  assumptions: [nenhuma, API estável]
  alternatives: [tratar com ValueError, retornar None]
  unknowns: nenhuma
  answered: true
simplify:
  minimum_code: true
  no_single_use_abstractions: true
  no_speculative_config: true
  answered: true
intent:
  code_does: dividir levanta ZeroDivisionError
  check_expects: teste espera ValueError
  spec_says: README não menciona o caso
  all_agree: true
  answered: true
surgical:
  files:
    - path: src/calculadora.py
      necessary: true
    - path: tests/test_calculadora.py
      necessary: true
  removed_imports_clean: true
  answered: true
verify:
  ruff_clean: true
  tests_pass: true
  coverage_pct: 100.0
  coverage_pass: true
  success_criteria_met: true
```

```

  attempts: 0
  hand_back: false
twins:
  pattern: a / b
  result: 1 arquivo(s), 1 ocorrência(s)
  searched: true
  defect_fixed: true
  matches_count: 1
  files_count: 1
  fix_applied: false
preflight:
  skills_missing: []
artifact:
  intent_owed: true
  intent_present: true
  auth_owed: false
  auth_present: false
  pending_owed: false
  pending_present: false
  twins_owed: true
  twins_present: true
base_commit: 762b73cb7343462235d08db2aba13e7f1022c392
approved_commit: 762
  b73cb7343462235d08db2aba13e7f1022c392

```

The task file is the contract between the cycle and the JUDGE. Every section is a claim that the JUDGE can confront with the repository: the verify section with re-execution, the surgical section with the diff, the intent section with the specification, the twins section with a pattern search. This is what makes the audit possible: a phase marked `answered: true` with empty content is a faked phase, and the audit names the concrete risk it created.

6 The adversarial layer: JUDGE

The JUDGE is the phase that no skill can perform. It treats the task file as a collection of claims and confronts them with the repository. It re-runs the claimed checks, diffs the claims against the actual changes, and hunts seven fraud categories:

- **weakened checks:** a test whose body was replaced by `pass`, an assertion turned into a comment, or a new test whose body is only `assert True` — it exists, it runs, it verifies nothing;
- **false completion:** the report claims lint clean, tests passing, and coverage at the gate, but the re-execution fails;
- **scope creep:** the report declares one file, the tree has four;
- **unauthorized action:** an irreversible action was taken without a documented AUTH line;
- **specification betrayal:** the intent gate recorded a disagreement and the task was approved anyway;
- **debris:** temporary files in the diff;
- **baseline tampering:** the `base_commit` declared in the YAML diverges from the Git anchor recorded when the task started.

6.1 Honest work: VERIFIED

After committing the task of Section 5, we ran `kata --task corrigir-divisao-zero --judge`. The judge re-executed the claimed checks and returned:

VEREDITO: VERIFIED

Claims verificadas:

- ruff check limpo (sem erros de lint)
- todos os testes passam
- coverage gate (100.0%)
- 2 arquivo(s) alterado(s) cirurgicamente (necessários)
- intenção alinhada: código, teste e spec concordam

Claims aceitas sem verificação (não re-executáveis):

- critério de sucesso satisfeito

Re-execução:

```

  lint
  teste
  coverage

```

Note that the success criterion is listed as *accepted without verification*: it is a subjective confirmation the user gives during VERIFY, and no command reproduces it. Presenting it as verified would be exactly the fraud the judge exists to hunt.

6.2 Fraudulent work: REFUTED

We then ran the judge against a trap scenario from the evaluation suite: a “completed” task whose report claims ruff clean, tests passing, and coverage at the gate, but where the re-execution fails all three (an unused import in the source, a failing test, and coverage skipped because pytest failed):

VEREDITO: REFUTED

Fraudes encontradas:

```

[high] false_completion
  ruff check re-executado falhou, mas relatório
  afirma que passou
  → relatório: ruff_clean=True → reality: ruff
  check falhou
[high] false_completion
  pytest re-executado falhou, mas relatório
  afirma que passou
  → relatório: tests_pass=True → reality: pytest
  falhou
[high] false_completion
  coverage re-executado falhou, mas relatório
  afirma que passou
  → relatório: coverage_pass=True → reality:
  coverage falhou

```

Re-execução:

```

  lint
  teste
  coverage

```

The verdict is REFUTED, and the exit code is 1. A skill would say “be honest”; Kata catches the lie.

6.3 Blind spots are confessed

The judge also confesses its blind spots. If it could not observe something — a test language it has no probes for (it knows Python, JS/TS, Go, Ruby, Rust, Java/Kotlin, C#, PHP, and Swift), a file outside Git’s diff, a baseline that no longer resolves — the verdict is UNVERIFIABLE, never VERIFIED. “I could not look” is never reported as “all clear”. With

fraud present, the fraud decides the verdict and the blind spots are still listed.

7 The audit mode

7.1 Audit grades and concrete risks

The audit mode (`kata --audit`) grades each phase of a task as followed, skipped, faked, or degraded, and names the concrete risk each skip or fake created. Table 3 summarizes the four grades and what each one means for the trustworthiness of the task.

Table 3. The four audit grades and their meaning.

Grade	Meaning
followed	The phase has answered: <code>true</code> and real content
skipped	The phase has skipped: <code>true</code> (documented, not hidden)
faked	The phase has answered: <code>true</code> but default or empty content
degraded	The phase ran without its own skill loaded (preflight)

The faked pattern (tracked in the project as issue R7-1) is a phase marked as answered without real content. This is only possible because there is objective logic behind it: the audit reads the YAML and detects `answered: true` with empty content. A skill has no way to know whether a phase was “faked”; Kata does.

```

FIT: faked
  rota e trivialidade não classificadas por humano -
  esforço pode ser
  desperdiçado em tarefa trivial ou mal roteada
THINK: faked
  assumptions nunca declaradas - qualquer solução
  pode atacar o
  problema errado
INTENT: faked
  código, teste e spec podem discordar sem registro
  - comportamento
  muda sem intenção verificada
SIMPLIFY: followed
SURGICAL: followed
VERIFY: followed

Audit encontrou 3 fake(s) e 0 skip(s).
```

The audit is the kata’s equivalent of the fable-method’s audit, and it catches phases that were filled in without being observed.

7.2 Preflight and installation checking

The orchestrator is not self-contained: at each phase it loads the matching skill and follows its instructions. When a skill is not installed, the call fails, the orchestrator has no instructions for that phase, and the model improvises from the phase name. The result is a task file with the section filled in and nothing behind it — precisely the faked phase the audit exists to catch, except produced by the tooling rather than by the agent, and invisible in the output.

The `kata --doctor` command reports, per frontend, which of the expected skills are present under the host’s configuration directory. A broken symlink does not count as installed, because `exists()` follows the link and so does the host when it tries to load the skill. A partial install is what fails, not an absent one: exit code 1 is reserved for a frontend that has some skills but not all, because someone with 9 of the 10 phase skills runs the whole cycle and silently loses a phase. With no frontend installed at all, `--doctor` says so and exits 0 — the CLI works without any. Domain adapters are optional and never fail `--doctor`: a missing `kata-devops` is reported as a hint because a coding task does not need it.

When a skill fails to load mid-cycle, the orchestrator is instructed not to improvise: it falls back to a per-phase minimum contract documented in the orchestrator itself, records the skill name in `preflight.skills_missing`, and discloses it in the report. The audit grades such a phase as degraded, not followed, because whatever the other grades read was written without those instructions. `VERIFY` and `JUDGE` degrade best, because their logic lives in the Python package rather than in the skill text.

8 Using the kata CLI

This section documents how to install Kata and use its command-line interface, so that the demonstrations of Sections 5–7 can be reproduced.

8.1 Installation

Kata requires Python 3.11 or newer. The package exposes a single entry point, `kata.cli:main`, installable in development mode with the lint, test, and coverage tools the CLI itself executes:

```
pip install -e '[dev]'
```

To run or install the CLI without touching the current environment, `pipx` or `uv` isolate the package in their own virtual environment:

```

# One-off run - nothing installed permanently (CI/
# headless friendly)
pipx run --spec . kata --version
uv tool run --from . kata --version

# Permanent install - creates a `kata` binary on the
# PATH
pipx install .
uv tool install . --force
```

The agent frontends are installed separately through symlinks. `make install` links the OpenCode agent and skills into the OpenCode configuration directory (honoring `OPENCODE_CONFIG_DIR`, defaulting to `~/.config/opencode`); `make install-claude-code` links the skills into the Claude Code configuration directory (honoring `CLAUDE_CONFIG_DIR`, defaulting to `~/.claude`). Because the installer creates symlinks, rebuilding the generated frontends with `make build-skills` is immediately visible without reinstalling.

8.2 Command reference

Table 4 lists the CLI modes. All modes operate on task files under `.kata/` at the target project's root; `--doctor` is the exception, checking skill installation without touching any task.

Verification is configurable per role. `--ruff-paths`, `--test-paths`, `--ignore`, `--cov-source`, and `--gate` set the built-in Python defaults (ruff, pytest, pytest-cov with `--cov-fail-under`, gate 70%). The 70% default is a project convention, not a research finding: each target project is expected to set its own gate via `--gate` or `.kata/config.yaml`, and the JUDGE compares the measured coverage against whatever gate the project declared. A role declared in `.kata/config.yaml` runs verbatim instead, so non-Python projects declare their own lint, test, and coverage commands; every undeclared role keeps its Python default.

8.3 Typical workflow and exit codes

A typical bug-fix session follows three commands. First, create the task and classify it (`kata --init fix-login`); second, apply the fix and run the full cycle (`kata --task fix-login`), answering each phase; third, verify adversarially before merging (`kata --task fix-login --judge`). For continuous integration, `kata --check-only` runs the objective checks non-interactively.

Exit codes make the CLI composable in scripts and CI pipelines. Exit code 0 means the requested operation passed (an approved cycle, a VERIFIED or caveated judge verdict, a clean audit, a complete install). Exit code 1 means failure: a rejected cycle, a REFUTED verdict, an audit with fakes or skips, or a partial skill installation. Exit code 2 means invalid CLI arguments. In particular, VERIFIED WITH CAVEATS and UNVERIFIABLE both exit 0: the judge verified what it could and approved with notes, or found no fraud but could observe nothing — treating either as failure would equate a caveat or a confessed blind spot with a high-severity fraud.

9 Research impact and evaluation

Kata is dogfooded: its own lint, tests, and coverage are measured (gate 70%), and the continuous integration runs the Makefile rather than restating the commands, so what is verified locally and remotely cannot drift apart.

The adversarial evaluation is the strongest evidence of impact: 19 trap scenarios (`eval/run_traps.py`) plant frauds the JUDGE must catch and honest work it must not accuse. Eleven scenarios plant a fraud that must be detected (weakened checks, false completion, scope creep, unauthorized action, specification betrayal, debris, baseline tampering); five scenarios expect a non-REFUTED verdict — four entirely honest tasks that must come back VERIFIED and one (s06) whose debris must be found while its look-alike names are spared (VERIFIED WITH CAVEATS) — and three expect UNVERIFIABLE for confessed blind spots. The trap suite includes a guard against refusing legitimate work: one scenario plants real debris beside files whose names merely look

like debris, and a judge that refuses honest work is treated as broken as one that misses fraud.

The unit suite also exercises the JUDGE against a real git repository in a temporary directory, because blindness to committed or untracked changes cannot be reproduced with mocks. The schema of the task file is compatible with the `.karpathy/` schema of the mushin agent (a private local project where the cycle was first implemented), so legacy tasks migrate through a symbolic link. The repository is public, MIT-licensed, and installable in both agent hosts through symlink installers.

9.1 Evaluation methodology

The evaluation of Kata follows the methodology of the Fable Method's eval Sahir619 (2026): adversarial trap scenarios that plant frauds the tool must catch, and honest scenarios it must not accuse. The trap suite is deliberately asymmetric in its coverage. Table 5 summarizes the 19 scenarios.

Twelve scenarios plant a fraud that must be detected, spanning all seven fraud categories: weakened checks (a test whose body was replaced by `pass` among real tests; an assertion turned into a comment; a new test whose body is only `assert True`; a fraudulent test left outside the index, invisible to `git diff`; a test body turned into `pass # comment`), false completion (a report claiming lint clean, tests passing, and coverage at the gate, where re-execution fails all three), scope creep (a report declaring one file when the tree has four), unauthorized action (an irreversible action without an AUTH line), specification betrayal (an intent gate that recorded disagreement and was approved anyway), debris (real temporary files in the diff), and baseline tampering (a `base_commit` rewritten after the Git anchor was recorded).

Four scenarios are entirely honest tasks that must come back VERIFIED. These are as important as the fraud scenarios: a judge that refuses legitimate work is as broken as one that misses fraud, and unit tests are poor at catching false positives because they test what the author thought to test. One honest scenario plants real debris beside files whose names merely look like debris (`templates/`, `temperature.py`, `attempt_parser.py`) — a judge that matches the substring “temp” would accuse honest work. Another honest scenario exercises the triviality gate with three small files and an empty surgical list, where a naive scope-creep rule would falsely accuse the task. A third exercises the `approved_commit` mechanism end to end: a later task touches an undeclared file after the approval, and the judge must not count it as scope creep for the earlier task. The remaining three scenarios expect UNVERIFIABLE for confessed blind spots: a test in a language the judge has no probes for (Elixir `.exs`), a task that claims no reproducible check at all, and a test hidden from Git via a local ignore rule that the judge cannot see in the diff.

The trap suite also guards the evaluation harness itself. One scenario opts into the real environment with `kata_visivel: true`, because the harness excluded `.kata/` from Git in every fixture and the judge counting the task's own file as scope creep survived ten review rounds — including the honest-work scenario whose whole job is catching false positives. A new scenario must be shown to

Table 4. The kata CLI modes.

Command	Effect
<code>kata --init TASK</code>	Create the task file, run FIT + THINK
<code>kata [--task TASK]</code>	Run the full interactive cycle
<code>kata --task TASK --plan</code>	Run FIT + THINK, save the plan, stop
<code>kata --check-only</code>	Run lint + tests + coverage only (CI entry)
<code>kata --task TASK --report</code>	Regenerate the outcome-first report
<code>kata --task TASK --judge</code>	Adversarial verification (hunts frauds)
<code>kata --task TASK --audit</code>	Grade phases: followed/skipped/faked
<code>kata --doctor</code>	Check phase-skill installation per frontend

Table 5. The 19 adversarial trap scenarios of the evaluation suite. Each row summarizes one `eval/scenarios/<name>/` directory, which pairs a `fixture/` (the planted fraud or honest work) with a `ground_truth.yaml` (the expected verdict and frauds). The scenario set is original to Kata, built incrementally across development sprints, each motivated by a real defect found in the JUDGE; the trap methodology follows the Fable Method’s eval Sahir619 (2026).

Scenario	What it exercises
s01	weakened checks: test body replaced by pass
s02	scope creep: report declares 1 file, tree has 4
s03	false completion: claims pass, re-execution fails all three
s04	unauthorized action: irreversible action without AUTH line
s05	spec betrayal: intent disagreement approved anyway
s06	debris + false positive guard: real debris beside look-alike names
s07	honest work: must come back VERIFIED
s08	weakened checks: fraudulent test left outside the index
s09	weakened checks: assertion turned into comment (modification diff)
s10	weakened checks: test body turned into pass # comment
s11	weakened checks: new test whose body is only <code>assert True</code>
s12	blind spot: test in Elixir <code>.exs</code> , no probes
s13	blind spot: task claims no reproducible check
s14	baseline tampering: <code>base_commit</code> rewritten after anchor
s15	honest work with <code>.kata/</code> visible to git
s16	honest trivial task with empty surgical list
s17	honest task with <code>approved_commit</code> end to end
s18	weakened checks: JS test turned into comment, <code>it.skip</code>
s19	blind spot: Git-ignored test via local exclude

fail when its defect is reintroduced, so a scenario that passes in both states protects nothing.

9.2 Trap scenario execution results

We executed the full trap suite (python3 `eval/run_traps.py`) on the version of the code described in this paper. All 19 scenarios passed: every planted fraud was detected with the expected verdict and the expected fraud types, and every honest scenario returned the expected verdict with no false accusations. Table 6 summarizes the outcome by expected verdict.

The result confirms the two-sided contract of the JUDGE. On the detection side, eleven scenarios returned REFUTED with high-severity frauds, covering all seven fraud categories; the s03 false-completion scenario shown in Section 6 is representative. On the restraint side, four honest scenarios returned VERIFIED, and the s06 scenario *also* returned VERIFIED WITH CAVEATS: the JUDGE found the real debris (`scratch/saida.tmp`, a debug print, a TODO) while sparing the look-alike names (`templates/`, `temperature.py`, `attempt_parser.py`) and the s16 trivial task that a naive

scope-creep rule would falsely accuse. Three scenarios returned UNVERIFIABLE, confessing blind spots instead of clearing work the judge could not observe. The suite runs in continuous integration on every push, so any regression in detection or restraint breaks the build.

9.3 Executed comparison with related gates

The strongest external evidence is the executed comparison of Section 3.3: four gates reviewed the same 19 scenarios, and Kata was the only one with zero false positives and a median review time of about one second. That comparison is a direct answer to the journal’s request for a software article to demonstrate its advance by comparison with available related software, and its raw verdicts are reproducible from the repository (`eval/compare_gates.py`, `eval/results/compare_gates_round2.json`).

10 Discussion

The research landscape of 2026 positions Kata at the intersection of three converging lines of work: agent self-reporting

Table 6. Trap suite execution results: 19/19 scenarios passed. Verdict counts derive from the `expected_verdict` fields of the 19 `ground_truth.yaml` files; the run reported here executed `python3 eval/run_traps.py` on the described code version, and the harness confirmed actual verdicts match expected ones with no false accusations.

Expected verdict	Scenarios	Count
REFUTED	s01, s02, s03, s04, s05, s08, s09, s10, s11, s14, s18	11
VERIFIED WITH CAVEATS	s06 (debris found; look-alike names spared)	1
VERIFIED	s07, s15, s16, s17	4
UNVERIFIABLE	s12, s13, s19	3

unreliability, benchmark measurement confounds, and claim-based verification.

First, the Unreliable Progress Bar study Wang et al. (2026) shows that agents cannot reliably report their own progress — the same unreliability that motivates Kata’s decision to never trust a claimed check without re-execution. The Double Measurement Confound Zhang et al. (2026) shows that agent benchmarks conflate scaffolding with capability; Kata’s trap suite avoids this by testing the tool, not the agent, and by measuring both fraud detection and false-positive avoidance.

Second, the claim-based verification model that Kata implements is emerging independently across domains: ReVerifier Mocan et al. (2026) verifies reference claims in manuscripts, the Discovery Certification Protocol Ning et al. (2026) certifies research-agent discoveries, and TrajMark Zeng et al. (2026) attributes coding-agent trajectories. Kata applies the same pattern to the software change itself, where the claims are the task file and the ground truth is the repository. This convergence suggests that claim-based verification is becoming a general pattern for trustworthy AI-assisted work, and Kata is a concrete instantiation for software engineering.

The executed comparison of Section 3.3 sharpens this positioning. All four gates detected every planted fraud, so the comparison separates *detection* from *restraint*: the difference showed up entirely on honest and blind-spot scenarios. The agent gates accused honest work (the baseline on s07), accused a later task’s changes as scope creep (all agents on s17) and reported false completion on blind spots the ground truth declares UNVERIFIABLE (s12, s13, s19). A gate that cannot see the mechanism — `approved_commit`, a Git-ignore rule, a language without probes — can only guess, and the guesses are what the comparison measured. Kata instead confesses the blind spot as UNVERIFIABLE. This is the same reliability-beyond-the-mean point of the Double Measurement Confound Zhang et al. (2026), applied to a change review rather than a benchmark.

Third, the limitations of Kata are honest and worth stating. Kata is a gate, not a silver bullet: it guarantees that what was claimed was verified, not that the solution is the best possible. It is interactive: the cycle asks a question at nearly every phase, and in headless mode the judgment phases use defaults and the report says explicitly that nobody answered. It requires Git for diff and branch detection. The success criterion is subjective, and the JUDGE accepts it without verification and says so in voice. The JUDGE has blind spots — test languages without probes, files outside the diff — and confesses them as UNVERIFIABLE rather than silencing them. Finally, the self-evaluation is smoke-test grade: the trap scenarios are deterministic fixtures, not a statistical study of

agent behavior, and the honest scenarios are few. The executed comparison of Section 3.3 mitigates this: it is the first external evaluation of Kata against related gates on identical workloads. A larger empirical study comparing Kata against a bare agent on real tasks, following the Fable Method’s cross-model methodology Sahir619 (2026), is future work.

11 Threats to validity

The evaluation of Kata, like any tool evaluation, is subject to threats to validity that we state explicitly.

Construct validity. The trap scenarios measure whether the JUDGE detects planted frauds and does not accuse honest work. The construct “fraud” is operationalized as the seven categories the JUDGE hunts, which are derived from the failure modes documented in the Fable Method Sahir619 (2026) and from the reward-hacking literature Zhao et al. (2026). A fraud category not in the list (for example, a new class of weakened test that the probes do not recognize) would not be detected, and the blind-spot mechanism is the honest response: the JUDGE confesses what it could not observe rather than claiming to have looked.

Internal validity. The trap scenarios are deterministic fixtures with a ground truth that specifies the exact verdict and the exact frauds the judge must find. The harness fails when the verdict differs, when a fraud is missing, or when an unexpected fraud appears. This determinism removes experimenter bias from the evaluation, but it also means the scenarios test what the authors thought to test. The honest scenarios mitigate this: they encode false positives that were actually observed in development (the debris false positive, the triviality-gate false positive, the `.kata/` visibility false positive), so they are regression tests for real defects rather than hypothetical ones.

External validity. The evaluation uses small synthetic fixtures, not real industrial tasks. The honest scenarios are few (four), and the cross-model methodology of the Fable Method Sahir619 (2026) — comparing the same task with and without the method across model tiers — has not been replicated for Kata. A larger empirical study on real tasks is future work. The tool itself, however, is used in production on the kata repository: its own changes go through the cycle, and the trap suite runs in continuous integration on every push.

Conclusion validity. The claim “the JUDGE catches fraud” is supported by the trap suite, but the claim “Kata improves the quality of AI-assisted changes” is not directly measured. The mechanism is plausible — forcing evidence before approval, re-executing claimed checks, and auditing completed work — and it is supported by the self-correction

literature Huang et al. (2024) and the reward-hacking literature Zhao et al. (2026), but a controlled study comparing changes made with and without Kata is needed to establish it.

12 Conclusion

Kata is not a skill — it is the tool that *executes* the discipline that skills ask for. The distinction is the thesis of this paper: a skill is an instruction; Kata is a tool that executes the instruction. The skill says “verify your tests”; Kata runs the tests, measures the coverage, compares it with the gate, and refuses to approve if it fails. The skill says “be honest”; the JUDGE re-runs the claimed checks and hunts seven categories of fraud.

The evaluation shows that the approach works. The trap suite closes on the JUDGE’s two-sided contract: every planted fraud is detected with the expected verdict and no honest scenario is accused. The executed comparison of Section 3.3 extends this beyond self-verification: over the same 19 scenarios, all four gates detected every fraud, but Kata was the only gate with zero false positives on honest work and a median review of about one second, while every agent gate accused at least two honest or blind-spot scenarios in tens of seconds per scenario. The limitations are honest: Kata is a gate, not a silver bullet — it guarantees that what was claimed was verified, not that the solution is the best possible; it is interactive (in headless mode the judgment phases use defaults and the report says explicitly that nobody answered); it requires Git; and the success criterion is subjective, which the JUDGE accepts without verification and says so in voice.

If you use AI agents to change code and want to know *for real* whether what was claimed is true, Kata is a concrete answer: it re-executes, compares, and refuses. The rest is conversation.

Acknowledgements

We thank the open-source communities of the Fable Method Sahir619 (2026) and of the Karpathy Development Cycle for the methodologies that inspired this work, and the maintainers of the related tools surveyed in Section 3 for the comparison that sharpened Kata’s design.

References

- Bouras, D. S., Dai, Y., and Mehtaev, S. (2026). Authority is not a string: A capability-scoped harness for prompt-injection-resistant coding agents.
- Hamidi, A., Konstantinou, M., Degiovanni, R., and Papadakis, M. (2026). How effective are traditional test criteria at detecting bugs in large language models generated code?
- Hartman, K. (2026). Introducing consort: A spec-first agent framework for enforced, test-driven development on live database branches.
- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. (2024). Large language models cannot self-correct reasoning yet. *International Conference on Learning Representations (ICLR)*.
- Huynh, H., Thongtanunam, P., Fu, M., Le, B., and Tantithamthavorn, K. (2026). Xagent: execution-guided agentic ai for effective localization and resolution of github issues.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. (2024). Swe-bench: Can language models resolve real-world github issues? *International Conference on Learning Representations (ICLR)*.
- LerianStudio (2026). Ring: Skills library for ai agents.
- Mocan, S., Angermeier, F., and Kreitz, M. (2026). Refverifier: Semi-automated reference claim verification for scientific manuscripts.
- Ning, J., Zhong, S., Li, X., and Zeng, J. (2026). Scores alone do not prove discovery: The discovery certification protocol for auditing ai research agents.
- Sahir619 (2026). The fable method: The fable workflow.
- TeamSPWK (2026). nova: Ai agent ops for claude code.
- vlad ko (2026). claude-wizard: Turn claude code from a fast coder into a senior software architect.
- Wang, B., Wang, Y., and Wu, Y. (2026). The unreliable progress bar: Can llm agents reliably report task progress throughout execution?
- Wang-Lin, X., Isopoussu, A., and Mahon, L. (2026). If it’s not buggy, don’t fix it: On the dynamics of iterative bug-fixing with llms.
- Zeng, B., Gao, Z., Li, X., Feng, X., and Jiang, J. (2026). Trajmark: Ownership attribution and segment-level tamper localization for coding-agent trajectories.
- Zhang, Y., Motaali, S., Dinh, V. P., Piroutiniya, A., de Vergara, J. E. L., de Pedro, L., Correia, R., Parra, I. M., and Xie, Y. (2026). The double measurement confound in agent benchmarks: De-scaffolding, ground-truth scoring, and reliability beyond the mean.
- Zhao, B., Srikanth, D., Wu, Y., and Jiang, Z. (2026). Specbench: Measuring reward hacking in long-horizon coding agents. *arXiv preprint arXiv:2605.21384*.